

Guia de Usuário

Simulação do Lançamento de um Foguete

Heloísa Rhodes, Paulo Vitor Misquita, Victor Gonçalves

Dezembro, 2023

1 Introdução

Este documento é um guia de usuário para o programa de simulação do lançamento de um foguete, desenvolvido por Heloísa Rhodes, Paulo Vitor Misquita e Victor Gonçalves para a disciplina de Introdução à Computação em Física. O objetivo deste guia é descrever o funcionamento do código do programa, bem como listar os pré-requisitos para executá-lo.

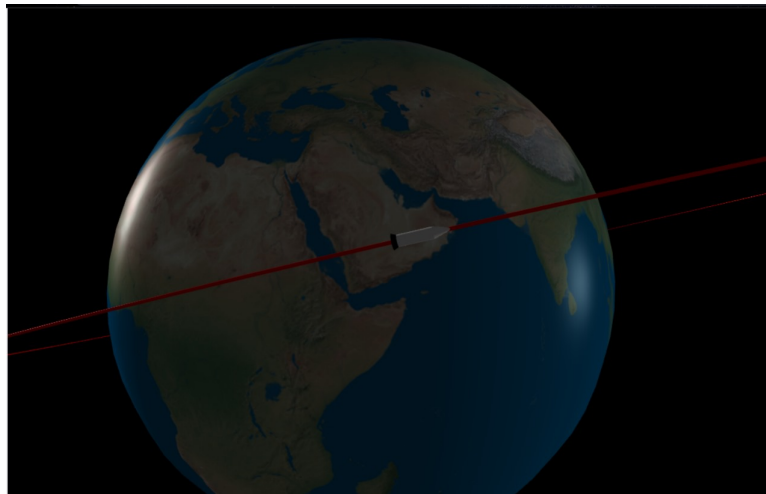


Figura 1: Simulação do lançamento de um foguete.

2 Pré-requisitos

O programa foi desenvolvido em Python, com o auxílio das bibliotecas Numpy, Matplotlib e VPython. As versões das bibliotecas, bem como do Python utilizadas foram:

- 1) Python - versão 3.11.4 64-bit
- 2) Numpy - versão 1.26.1
- 3) Matplotlib - versão 3.8.0
- 4) VPython - versão 7.6.4

3 Como o código funciona?

O programa foi dividido em duas partes: o código responsável pela realização dos cálculos e o código da simulação.

3.1 Cálculos

O código dos cálculos também foi dividido em duas partes: uma para o cálculo das posições durante o lançamento e outra para o cálculo da órbita do foguete.

Inicialmente, são definidas as constantes e variáveis iniciais (figura 2). São elas:

1. Massa molar do ar
2. Constante Universal dos Gases Ideais
3. Massa inicial do foguete
4. Área frontal do foguete
5. Coeficiente de Arrasto
6. Constante da Gravitação Universal
7. Massa da Terra
8. Raio da Terra
9. Área de Exaustão do foguete
10. Ângulo de voo inicial
11. Pressão no nível do mar
12. Temperatura do ar no nível do mar
13. Taxa de queima de combustível
14. Velocidade de exaustão
15. Pressão de exaustão
16. Tempo total de simulação

```
#Constantes
R = 8.314                                     #Constante Universal dos Gases Ideais (J/kgK)
G = 6.67e-11                                #Constante de Gravitação universal (Nm²/kg²)

#Dados da Terra
Mt = 5.98e24                                #Massa da terra (kg)
Re = 6371000.0                              #Raio da Terra (m)

#Dados da atmosfera
M = 0.02896                                 #Massa molar do ar (kg/mol)
P0 = 101325.0                               #Pressão Atmosférica no nível do mar (Pa)
T0 = 300.0                                  #Temperatura da atmosfera no nível do mar (K)

#Dados do foguete
m = 2990000.0                               #Massa inicial do foguete (kg)
A = 25*np.pi                               #Área frontal do foguete (m²)
C = 0.3                                     #Coeficiente de arrasto
Ae = 8.82                                   #Área de exaustão (m²)
me = 10040.0                               #Taxa de queima de combustível (kg/s)
ve = 2256.0                                #Velocidade de exaustão (m/s)
Pe = 400000.0                              #Pressão de exaustão (Pa)
y0 = 89.999*np.pi/180                    #Ângulo de voo inicial (rad)
v0 = 0.0                                    #Velocidade inicial (m/s)
h0 = 0.0                                    #Altura inicial (m)
x = 0.0                                    #Posição x inicial (m)
```

Figura 2: Declaração de variáveis.

Então, são definidas as funções que calculam as variáveis de ambiente: gravidade, pressão atmosférica e densidade da atmosfera. Para o cálculo da pressão e densidade da atmosfera, foram definidas duas funções: uma para uma faixa de até 11km, e outra para além desta altitude.

Em seguida, foram definidas as funções com as equações do foguete, para a taxa de variação da velocidade, da posição em relação à superfície da Terra, da altura e do ângulo de caminho de voo, todas em relação ao tempo (figura 3). Foram criadas, também, listas preenchidas com zeros, com a dimensão da quantidade total de valores necessários para a simulação, para armazenar os valores das variáveis, que posteriormente serão utilizadas para plotar gráficos e escrever arquivos (figura 4).

```
#Definição de funções
#Arrasto dividido pelo quadrado da velocidade
def arrasto(dc):
    return (dc*A*15)/2

#Gravidade
def gravidade(hc):
    return G*Mt/(Re+hc)**2

#Densidade da atmosfera na primeira faixa (até 11km de altitude)
def dens1(P0,g,T0,h):
    d=(P0*M/(R*T0))*(1+2*M*g*(-h)/(7*R*T0))**(5/2)
    return(d)

#Pressão atmosférica na primeira faixa (até 11km de altitude)
def press1(P0,g,T0,h):
    P=P0*(1+2*M*g*(-h)/(7*R*T0))**(7/2)
    return(P)

#Densidade da atmosfera na segunda faixa (após 11km de altitude)
def dens2(P1,g,T1,h):
    d=(P1*M/(R*T1))*np.exp(-M*g*h/(R*T1))
    return(d)

#Pressão atmosférica na segunda faixa (após 11km de altitude)
def press2(P1,g,T1,h):
    P=P1*np.exp(-M*g*h/(R*T1))
    return(P)
```

Figura 3: Declaração de funções.

```
#Criando vetores preenchidos com zeros, de dimensão igual ao tempo total dividido pelo intervalo de tempo da iteração
n = int(time/dt)
x_pos = np.zeros(n)           #Posição x
y_pos = np.zeros(n)           #Posição y (altitude)
V_pos = np.zeros(n)           #Ângulo de voo
t_pos = np.zeros(n)           #Instante de tempo
m_pos = np.zeros(n)           #Massa do foguete
E_pos = np.zeros(n)           #Anomalia excêntrica
exc_pos = np.zeros(n)         #Excentricidade
a_pos = np.zeros(n)           #Semieixo maior
p_pos = np.zeros(n)           #Posição na órbita
Re_pos = np.array([Re]*n)     #Lista preenchida com o valor do raio da Terra (apenas para referência)
```

Figura 4: Criando listas preenchidas com zeros.

Após definidas as funções, foi iniciado o laço de iteração que calcula os valores das funções utilizando métodos numéricos. O laço começa verificando o tempo total decorrido, a fim de determinar o estágio do foguete e ajustar as variáveis de acordo com cada estágio.

O método numérico que será utilizado é o método de Euler Melhorado, o que implica que é necessário saber os valores das funções em um ponto anterior, a fim de calcular o próximo. As funções utilizadas fornecem as taxas de variações das grandezas no tempo, no entanto, essas funções não possuem o tempo como variável independente de maneira direta. Assim, para poder utilizar o método, foi necessário inicialmente determinar os valores das funções após um pequeno intervalo de tempo e utilizar esse valor para calcular os próximos. Esse passo foi feito utilizando o método de Euler Simples e as condições iniciais. Com o primeiro ponto calculado, foi retornado um passo na iteração e aplicado novamente o método de Euler simples para calcular um segundo ponto, agora partindo dos dados do primeiro ponto. Com os dois pontos calculados é possível aplicar o método de Euler Melhorado, o que é feito logo em seguida (figura 5).

Dentro do laço também é verificada a altitude do foguete, com o objetivo de determinar em qual faixa da atmosfera ele está e utilizar as funções corretas para calcular a densidade da atmosfera e a pressão atmosférica. O laço de iteração que realiza os cálculos do lançamento acontece até que

```

#Aplicação do método de Euler Melhorado
v_back=dv_dt(v,P,D,g,Y,m,Pe,ve,me)
v=v1
v1+=(v_back+dv_dt(v+v_back*dt,P1,D1,g1,Y1,m1,Pe,ve,me))*dt/2
X=X1
X1+=(Re/(Re+h))*v*np.cos(Y)+Re/(Re+h1)*v1*np.cos(Y1)*dt/2
x_pos[int(t/dt)]=X1
h=h1
h1+=(v*np.sin(Y))+v1*np.sin(Y1)*dt/2
y_pos[int(t/dt)]=h1
Y_back=dy_dt(v,h,g,Y)
Y=Y1
Y1=(dv2_dt(Y,t)+dv2_dt(Y+dt*dv2_dt(Y,t),t+dt))*dt/2
Y_pos[int(t/dt)]=Y1*180/np.pi
m=m1
m1-=me*dt
m_pos[int(t/dt)]=m
t_pos[int(t/dt)]=t
E_pos[int(t/dt)]=v**2/2-G*Mt/(h+Re)
e_m=v**2/2-G*Mt/(h+Re)
exc_pos[int(t/dt)]=((1+(2*e_m*((h+Re)*v*np.sin(np.pi/2+Y))**2)/(G*Mt)**2)**0.5)
a_pos[int(t/dt)]=-G*Mt/(2*e_m)
p_pos[int(t/dt)]=a_pos[int(t/dt)]*(1-exc_pos[int(t/dt)])
t+=dt

```

Figura 5: Código que executa o método de Euler Melhorado.

tenha se passado um tempo total, definido pela variável *time*, e o laço utiliza um passo determinado pela variável *dt*, que deve possuir um valor pequeno, para a aplicação dos métodos numéricos. Após atingido o tempo total, que deve ser o tempo necessário para o foguete poder entrar em órbita, o laço finaliza e os cálculos do lançamento estão finalizados.

Finalizados os cálculos, são plotados gráficos das grandezas, a fim de observar o comportamento obtido. Após isso, são criados arquivos de texto para armazenar os valores das posições em x e em y, que estão em listas (figura 6). Com os arquivos criados, a parte do código que calcula o lançamento é finalizada.

```

#Criando os arquivos com as posições
print("Criando arquivos do lançamento")

#Arquivo com as posições em x
print("Criando arquivo das posições x")
with open("posicoesXLancamento.txt","w") as file:
    indxc = len(x_pos)
    for ka in range(0,indxc):
        file.write(str(x_pos[ka])+",")
print("Arquivo das posições x criado")

print("Criando arquivo das posições y")
#Arquivo com as posições em y
with open("posicoesYLancamento.txt","w") as file:
    indyc = len(y_pos)
    for ka in range(0,indyc):
        file.write(str(y_pos[ka])+",")
print("Arquivo das posições y criado")

print("Arquivos do lançamento criados")
print("Etapa do Lançamento finalizada\n")

```

Figura 6: Código que cria os arquivos das posições do lançamento.

Então o programa parte para o cálculo da órbita. O funcionamento é semelhante, começa com a declaração das funções que serão utilizadas e da criação de listas preenchidas com zeros, com a dimensão igual a quantidade total de dados necessária para os cálculos. Então é calculado os valores das funções em um primeiro ponto, utilizando as condições iniciais, e, a partir disso, começa o laço que realiza os cálculos da órbita. Aqui é utilizado o método de Newton. Da mesma forma que o lançamento, o laço de cálculo da órbita ocorre até um tempo total, determinado pela variável *time2*, que define o tempo total para o qual será calculada a órbita, que foi utilizado o tempo necessário para o foguete realizar uma volta completa em torno da Terra. O laço encerra quando é atingido esse tempo.

Finalizado o cálculo da órbita, novamente são plotados gráficos para observar o comportamento. Após isso, são criados dois arquivos de texto para armazenar as posições da órbita, um para as posições em x e o outro para as posições em y. Com os quatro arquivos criados, o código dos cálculos é finalizado e o processo para realizar a simulação está pronto.

3.2 Simulação

As posições calculadas são em relação à superfície da Terra, assim, para a simulação é necessário considerar a curvatura da superfície terrestre. Para isso são declaradas funções que realizam essa transformação. Em seguida, são criados os modelos da Terra e do foguete.

Então, o programa lê os arquivos que contêm os valores das posições e os armazena em listas. Para a animação, é realizado um laço (figura 7) que atualiza a posição do foguete a partir dos valores das listas, levando em conta o efeito da curvatura da Terra, ou seja, aplicando a função que realiza a conversão. Durante o laço é atualizado também o eixo do modelo do foguete. O laço dura o tempo determinado pelos dados dos cálculos realizados, terminado o laço do lançamento, é executado o laço da órbita. Os dados da órbita foram calculados para uma volta em torno da Terra, dessa forma, esse laço é repetido para o foguete realizar mais de uma volta em órbita. A simulação é encerrada fechando a janela de execução.

Durante a simulação, existe uma opção de *Track* disponível, que, quando ativa, acompanha o foguete com a câmera.

```
#Laço do lançamento
for i in range(0,int(time_launch/100)):
    rate(1000)

    #Mudança do sistema de coordenadas
    alfa=angulo(float(posicoesX[i*100]))
    x=turnx(alfa,float(posicoesY[i*100])+Re)
    y=turny(alfa,float(posicoesY[i*100])+Re)

    #Atualização da posição do foguete
    proj.pos = vector(x,0,y)
    nozzle.pos = vector(x,0,y+200000)
    base.pos = vector(x,0,y)

    #Rotação do foguete, a partir do segundo índice pois é preciso uma posição anterior
    if(i>0):
        #Deslocamentos em relação ao eixos coordenados
        Vx=x-turnx(alfa,float(posicoesY[i*100-1]))
        Vy=y-turny(alfa,float(posicoesY[i*100-1]))

        #Aproximação dos senos e cossenos entre o vetor velocidade e os eixos coordenados
        axz=Vy/(Vx**2+Vy**2)**0.5
        axx=Vx/(Vx**2+Vy**2)**0.5

        #Atualização do eixo do foguete e de sua rotação
        proj.axis=vector(200000*axz,0,-200000*axx)
        nozzle.axis=vector(100000*axz,0,-100000*axx)
        base.axis=vector(100000*axz,0,-100000*axx)
        nozzle.pos=vector(x+200000*axz,0,y-200000*axx)
```

Figura 7: Laço que realiza a simulação do lançamento.